

```
def findMaxLength(nums):
```

```
    prefix_map = {0: -1}
```

Recursive

```
    def helper(i, prefix_sum, max_len):
```

```
        if i == len(nums):
```

```
            return max_len →  $C_1$ 
```

```
        if nums[i] == 1:
```

```
            prefix_sum += 1
```

```
        else:
```

```
            prefix_sum -= 1 →  $C_2$ 
```

```
        if prefix_sum in prefix_map:
```

```
            max_len = max(max_len, i - prefix_map[prefix_sum])
```

```
        else:
```

```
            prefix_map[prefix_sum] = i →  $C_3$ 
```

```
        return helper(i + 1, prefix_sum, max_len)  $T(n-1)$ 
```

```
    return helper(0, 0, 0)
```

$$T(n) = T(n-1) + C$$

$$T(n-1) = T(n-2) + C \implies T(n) = T(n-2) + 2C$$

$$T(n-2) = T(n-3) + C \implies T(n) = T(n-3) + 3C$$

$$T(n) = T(n-k) + kC \quad \#$$

Base Case $\therefore T(0) = \theta(1) \implies n - k = 0 \implies k = n$

$$T(n) = \theta(1) + nC \implies T(n) \in \theta(n)$$

```

def findMaxLength(nums):
    prefix_map = {0: -1}
    prefix_sum = 0
    max_len = 0

    for i, num in enumerate(nums):
        if num == 1: —————> Basic operation 1
            prefix_sum += 1
        else:
            prefix_sum += -1

        if prefix_sum in prefix_map: —————> Basic operation 2
            max_len = max(max_len, i - prefix_map[prefix_sum])
        else:
            prefix_map[prefix_sum] = i

    return max_len

print(findMaxLength(arr))

```

$$C(n) = \sum_{i=0}^{n-1} 1 = n - 1 + 1 = \boxed{n}$$

$$T(n) = T_{b_0} \cdot C(n)$$

• basic operation $\Rightarrow$ 2 if conditions

• let time of basic operation (T_{b_0}) = 1

$$T(n) = C(n) \Rightarrow T(n) \in \theta(n)$$

FUNCTION findMaxLength(nums):

Recursive

CREATE hashMap with {0: -1}

FUNCTION helper(i, prefix_sum, max_len):

IF i equals length of nums:

RETURN max_len ← base case, stop here

IF nums[i] is 1:

ADD 1 to prefix_sum

ELSE:

SUBTRACT 1 from prefix_sum

IF prefix_sum is in hashMap :

length = i - hashMap[prefix_sum]

max_len = maximum of (max_len, length)

ELSE:

WRITE prefix_sum → i in hashMap

RETURN helper(i+1, prefix_sum, max_len) ← call itself

RETURN helper(0, 0, 0) ← start from index 0

FUNCTION findMaxLength(nums):

non-Rec

CREATE hashMap with {0: -1}

SET prefix_sum = 0

SET max_len = 0

FOR every index i and element num in nums:

IF num is 1:

ADD 1 to prefix_sum

ELSE:

SUBTRACT 1 from prefix_sum

IF prefix_sum is in notebook:

length = i - hashMap[prefix_sum]

max_len = maximum of (max_len, length)

ELSE:

WRITE prefix_sum → i in hashMap

RETURN max_len

- $0 \rightarrow -1$
- $1 \rightarrow +1$

EX list = [0, 1, 1, 0,]

- PremaP = {0: -1}
- PreSum = 0, • MaxLength = 0

- 0) num = 0 $\Rightarrow 0 - 1 = -1 \Rightarrow$ PremaP does not have (-1) Then PremaP = {0: -1, -1: 0}
- 1) num = 1 $\Rightarrow -1 + 1 = 0 \Rightarrow$ PremaP does have 0 Then $\text{maxLength} = 1 - (-1)$
- 2) num = 1 $\Rightarrow 0 + 1 = 1 \Rightarrow$ PremaP does not have 1 Then PremaP = {0: -1, -1: 0, 1: 2}
- 3) num = 0 $\Rightarrow 1 - 1 = 0 \Rightarrow$ PremaP does have 0 Then $\text{maxLength} = 3 - (-1) = 4$

TYPE :	Iterative	Recursive
Time Complexity	$\theta(n)$	$\theta(n)$
Space Complexity	$\theta(n)$	$\theta(n) + \theta(n)$ of call stack
Memory & Performance	Less , more	more , Less
Complexity	Easy to understand	Not easy to understand